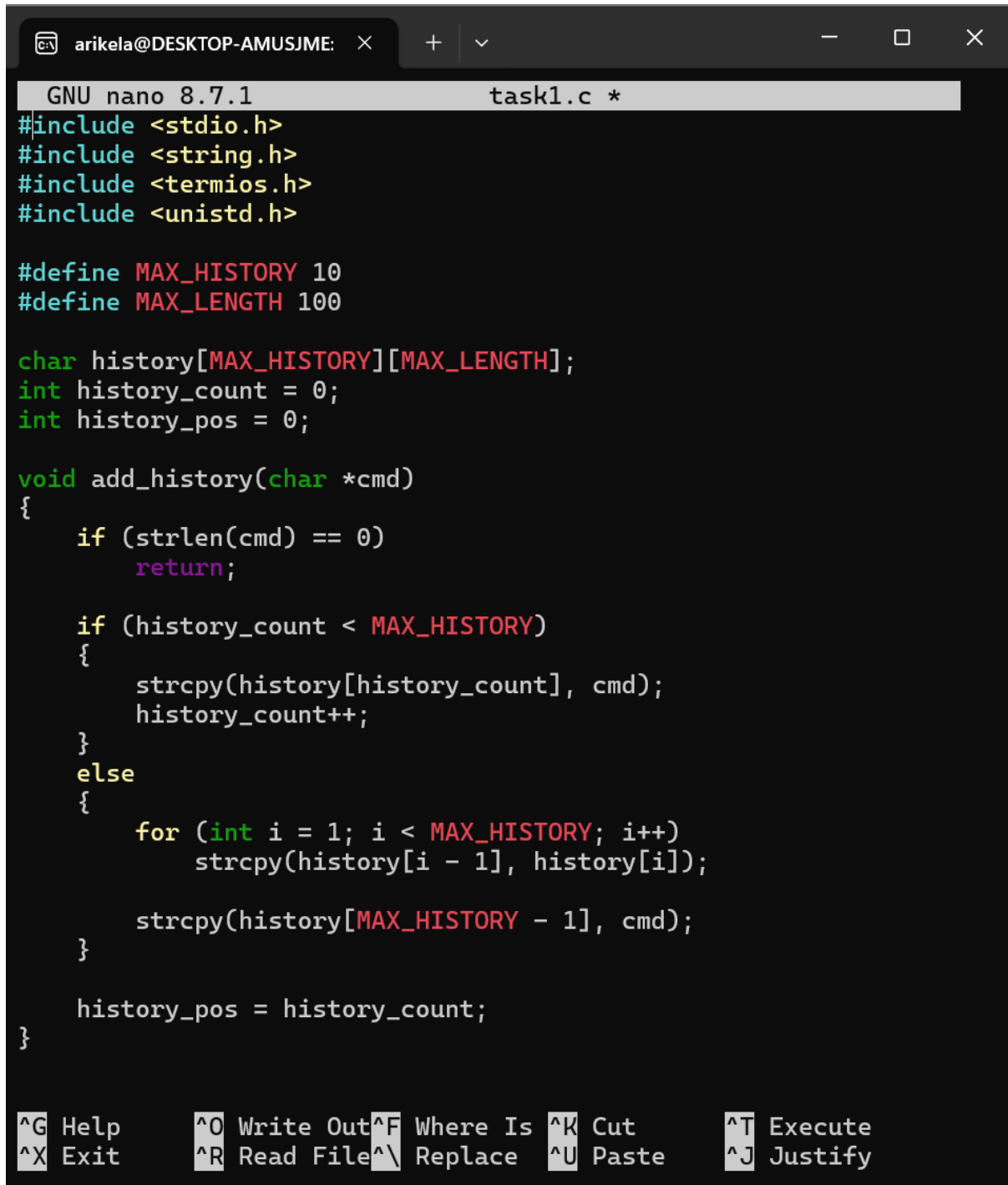


Skill Week -3

Task-1

To Apply Escape Sequences, Store Command History, Navigate Previous Commands, Navigate Next Commands, Update Input Buffer, Test Recall Functionality



```
GNU nano 8.7.1 task1.c *
#include <stdio.h>
#include <string.h>
#include <termios.h>
#include <unistd.h>

#define MAX_HISTORY 10
#define MAX_LENGTH 100

char history[MAX_HISTORY][MAX_LENGTH];
int history_count = 0;
int history_pos = 0;

void add_history(char *cmd)
{
    if (strlen(cmd) == 0)
        return;

    if (history_count < MAX_HISTORY)
    {
        strcpy(history[history_count], cmd);
        history_count++;
    }
    else
    {
        for (int i = 1; i < MAX_HISTORY; i++)
            strcpy(history[i - 1], history[i]);

        strcpy(history[MAX_HISTORY - 1], cmd);
    }

    history_pos = history_count;
}

^G Help      ^O Write Out ^F Where Is  ^K Cut       ^T Execute
^X Exit      ^R Read File ^\ Replace   ^U Paste     ^J Justify
```

```
    history_pos = history_count;
}

int main()
{
    struct termios oldt, newt;
    char input[MAX_LENGTH];
    char c;

    printf("=== COMMAND HISTORY DEMONSTRATION ===\n");
    printf("Enter commands. Use UP/DOWN arrows to navigate.\n");
    printf("Press 'q' to quit.\n\n");

    tcgetattr(STDIN_FILENO, &oldt);
    newt = oldt;

    newt.c_lflag &= ~(ICANON | ECHO);
    tcsetattr(STDIN_FILENO, TCSANOW, &newt);

    while (1)
    {
        printf("\n$ ");
        fflush(stdout);

        int index = 0;
        memset(input, 0, MAX_LENGTH);

        while (1)
        {
            c = getchar();

            /* Quit */
```

```
^G Help
^X Exit
```

```
^O Write Out
^R Read File
```

```
^F Where Is
^N Replace
```

```
^K Cut
^U Paste
```

```
^T Execute
^J Justify
```



arikela@DESKTOP-AMUSJME: ×



GNU nano 8.7.1

task1.c *

```
/* Quit */
if (c == 'q' && index == 0)
{
    printf("\nExiting...\n");
    tcsetattr(STDIN_FILENO, TCSANOW, &oldt);
    return 0;
}

/* Enter */
if (c == '\n')
{
    input[index] = '\0';

    if (strlen(input) > 0)
    {
        add_history(input);
        printf("\nCommand stored: %s\n", input);
    }

    break;
}

/* Escape sequence */
if (c == 27)
{
    char c1 = getchar();
    char c2 = getchar();

    if (c1 == '[' && c2 == 'A')
    {
        /* UP arrow */
        if (history_count > 0 && history_pos > 0)
        {
            |
```

^G Help

^X Exit

^O Write Out

^R Read File

^F Where Is

^_ Replace

^K Cut

^U Paste

^T Execute

^J Justify

arikela@DESKTOP-AMUSJME: ×

+ ▾

—

□

×

GNU nano 8.7.1

task1.c *

```
        /* UP arrow */
        if (history_count > 0 && history_pos > 0)
        {
            history_pos--;
            strcpy(input, history[history_pos]);

            printf("\r$ %-99s", input);
            fflush(stdout);
        }
    }
    else if (c1 == '[' && c2 == 'B')
    {
        /* DOWN arrow */
        if (history_pos < history_count - 1)
        {
            history_pos++;
            strcpy(input, history[history_pos]);

            printf("\r$ %-99s", input);
            fflush(stdout);
        }
        else
        {
            history_pos = history_count;
            memset(input, 0, MAX_LENGTH);

            printf("\r$ %-99s", "");
            fflush(stdout);
        }
    }

    continue;
}
```

^G Help
^X Exit

^O Write Out
^R Read File

^F Where Is
^_ Replace

^K Cut
^U Paste

^T Execute
^J Justify

arikela@DESKTOP-AMUSJME: ×

+ ▾

—

□

×

GNU nano 8.7.1

task1.c *

```
        /* DOWN arrow */
        if (history_pos < history_count - 1)
        {
            history_pos++;
            strcpy(input, history[history_pos]);

            printf("\r$ %-99s", input);
            fflush(stdout);
        }
        else
        {
            history_pos = history_count;
            memset(input, 0, MAX_LENGTH);

            printf("\r$ %-99s", "");
            fflush(stdout);
        }
    }

    continue;
}

/* Normal character */
if (index < MAX_LENGTH - 1)
{
    input[index++] = c;
    putchar(c);
    fflush(stdout);
}
}
}
|
```

^G Help

^X Exit

^O Write Out

^R Read File

^F Where Is

^ Replace

^K Cut

^U Paste

^T Execute

^J Justify

```
arikel@DESKTOP-AMUSJME:~/OSSP$ nano task1.c
arikel@DESKTOP-AMUSJME:~/OSSP$ gcc task1.c -o task1
arikel@DESKTOP-AMUSJME:~/OSSP$ ./task1
=== COMMAND HISTORY DEMONSTRATION ===
Enter commands. Use UP/DOWN arrows to navigate.
Press 'q' to quit.

$ ls
Command stored: ls

$ pwd
Command stored: pwd

$ date
Command stored: date

$ |
```

Task-2

To Allocate Buffers Dynamically, Resize Arrays, Prevent Buffer Overflow, Manage Linked Lists, Release Memory Correctly, Verify with Valgrind

GNU nano 8.7.1

task2.c *

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

typedef struct Node {
    char *command;
    struct Node *next;
} Node;

int main()
{
    int capacity = 2;
    int count = 0;

    /* Dynamically allocate array */
    char **commands = malloc(capacity * sizeof(char *));

    if (commands == NULL) {
        printf("Memory allocation failed!\n");
        return 1;
    }

    printf("=== DYNAMIC MEMORY MANAGEMENT ===\n\n");

    char *data[] = {
        "ls",
        "pwd",
        "date",
        "echo Hello"
    };

    /* Add commands and resize when necessary */
    for (int i = 0; i < 4; i++) {
```

 ^G Help ^X Exit ^O Write Out ^R Read File ^F Where Is ^\ Replace ^K Cut ^U Paste ^T Execute ^J Justify

```
/* Add commands and resize when necessary */
for (int i = 0; i < 4; i++) {

    if (count == capacity) {
        capacity *= 2;

        char **temp = realloc(commands,
                                capacity * sizeof(char *));

        if (temp == NULL) {
            printf("Memory reallocation failed!\n");
            return 1;
        }

        commands = temp;

        printf("Array resized. New capacity: %d\n", capacity);
    }

    commands[count] = malloc(strlen(data[i]) + 1);

    if (commands[count] == NULL) {
        printf("Memory allocation failed!\n");
        return 1;
    }

    strcpy(commands[count], data[i]);
    count++;
}

printf("\nDynamic Array:\n");

for (int i = 0; i < count; i++)
```

 ^G Help ^X Exit ^O Write Out ^R Read File ^F Where Is ^\ Replace ^K Cut ^U Paste ^T Execute ^J Justify

arikela@DESKTOP-AMUSJME: ×

+ v

— □ ×

GNU nano 8.7.1

task2.c *

```
for (int i = 0; i < count; i++)
    printf("%d. %s\n", i + 1, commands[i]);

/* Create linked list */
Node *head = NULL;

for (int i = 0; i < count; i++) {

    Node *new_node = malloc(sizeof(Node));

    if (new_node == NULL)
        return 1;

    new_node->command = malloc(strlen(commands[i]) + 1);

    if (new_node->command == NULL)
        return 1;

    strcpy(new_node->command, commands[i]);

    new_node->next = head;
    head = new_node;
}

printf("\nLinked List:\n");

Node *current = head;

while (current != NULL) {
    printf("-> %s\n", current->command);
    current = current->next;
}
```

^G Help
^X Exit

^O Write Out
^R Read File

^F Where Is
^_ Replace

^K Cut
^U Paste

^T Execute
^J Justify

```
}

printf("\nLinked List:\n");

Node *current = head;

while (current != NULL) {
    printf("-> %s\n", current->command);
    current = current->next;
}

/* Free dynamic array memory */
for (int i = 0; i < count; i++)
    free(commands[i]);

free(commands);

/* Free linked list memory */
current = head;

while (current != NULL) {
    Node *temp = current;
    current = current->next;

    free(temp->command);
    free(temp);
}

printf("\nAll allocated memory released successfully.\n");

return 0;
}
```

^G Help
^X Exit

^O Write Out
^R Read File

^F Where Is
^_ Replace

^K Cut
^U Paste

^T Execute
^J Justify

```
arikel@DESKTOP-AMUSJME:~/OSSP$ nano task2.c
arikel@DESKTOP-AMUSJME:~/OSSP$ gcc -Wall -Wextra -g task2.c -o
task2
arikel@DESKTOP-AMUSJME:~/OSSP$ ./task2
=== DYNAMIC MEMORY MANAGEMENT ===

Array resized. New capacity: 4

Dynamic Array:
1. ls
2. pwd
3. date
4. echo Hello

Linked List:
-> echo Hello
-> date
-> pwd
-> ls

All allocated memory released successfully.
```

arikela@DESKTOP-AMUSJME: X

+ v

-

□

X

```
arikela@DESKTOP-AMUSJME:~/OSSP$ valgrind --leak-check=full ./task2
```

```
==1258== Memcheck, a memory error detector
==1258== Copyright (C) 2002-2024, and GNU GPL'd, by Julian Seward et al.
```

```
==1258== Using Valgrind-3.26.0 and LibVEX; rerun with -h for copyright info
```

```
==1258== Command: ./task2
```

```
==1258==
```

```
=== DYNAMIC MEMORY MANAGEMENT ===
```

```
Array resized. New capacity: 4
```

```
Dynamic Array:
```

1. ls
2. pwd
3. date
4. echo Hello

```
Linked List:
```

- > echo Hello
- > date
- > pwd
- > ls

```
All allocated memory released successfully.
```

```
==1258==
```

```
==1258== HEAP SUMMARY:
```

```
==1258==      in use at exit: 0 bytes in 0 blocks
```

```
==1258==    total heap usage: 15 allocs, 15 frees, 1,182 bytes allocated
```

```
==1258==
```

```
==1258== All heap blocks were freed -- no leaks are possible
```

```
==1258==
```

```
==1258== For lists of detected and suppressed errors, rerun with: -s
```

```
==1258== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0
```